

Lecture # 9

Recursion

- The programs we have discussed in previous programming courses are generally structured as functions that call one another in a disciplined, hierarchical manner.
- For some problems, it is useful to have functions call themselves.
- A *recursive function* is a function that **calls itself** either directly or indirectly through another function.

- A recursive function is called to solve the problem. The function actually knows how to solve the simplest case (s) or so called Base Case (s).
- If the function is called with a base case, the function simply returns the result.
- If a function is called with more complex problem, the function divides the problem into two conceptual pieces:
 - A piece the function knows how to do and
 - A piece the function doesn't know how to do.

- To make recursion feasible the latter piece (i.e. piece the function doesn't know how to do) must resemble the original problem, but in the form of slightly simpler or slightly smaller version of the original problem.
- Because this new problem looks like the original problem the function **launches (calls)** a **fresh (new) copy** of itself to go to work on smaller problems. This is referred to as a recursive call and is also called *recursion step*.

Example : Factorial

- Given a positive integer n , n factorial is defined as the product of all integers between n and 1 . for example...

$$5! = 5 * 4 * 3 * 2 * 1 = 120$$

$$0! = 1$$

so

$$n! = 1 \text{ if } (n == 0)$$

$$n! = n * (n-1) * (n-2) * (n-3) \dots * 1 \text{ if } n > 0$$

- So we can present an algorithm that accepts integer **n** and returns the value of **n!** as follows.

```
int prod = 1, x, n;  
cin>>n;  
for( x = n; x > 0; x-- )  
    prod = prod * x;  
cout<<n<<" factorial is"<<prod;
```

- Such an algorithm is called *iterative* algorithm because it calls itself for the explicit (precise) repetition of some process until a certain condition is met.

- In above program $n!$ is calculated like as follows.

$$0! = 1$$

$$1! = 1$$

$$2! = 2 * 1$$

$$3! = 3 * 2 * 1$$

$$4! = 4 * 3 * 2 * 1$$

- Let us see how the recursive definition of the factorial can be used to evaluate 5!.

1. $5! = 5 * 4!$
2. $4! = 4 * 3!$
3. $3! = 3 * 2!$
4. $2! = 2 * 1!$
5. $1! = 1 * 0!$
6. $0! = 1$

- In above each case is reduced to a simpler case until we reach the case 0! Which is defined directly as 1. we may therefore backtrack from line # 6 to line # 1 by returning the value computed in one line to evaluate the result of previous line.

Lets incorporate this previous process of backward going into an algorithm as follows.....

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
int factorial( int numb )
```

```
{
```

```
    if( numb <= 0 )
```

```
        return 1;
```

```
    else
```

```
        return numb * factorial( numb - 1 );
```

```
}//-----
```

```
void main()
```

```
{
```

```
    clrscr();
```

```
    int n;
```

```
    cout<<" \n Enter no for finding its Factorial.\n";
```

```
    cin>>n;
```

```
    cout<<"\n Factorial of "<<n<<" is : "<<factorial( n );
```

```
    getch();
```

```
}// end of main().
```

- Above code reflects recursive algorithm of $n!$. In this function is first called by `main( )` and then it calls `itself` until it calculates the $n!$ and finally return it to `main( )`.

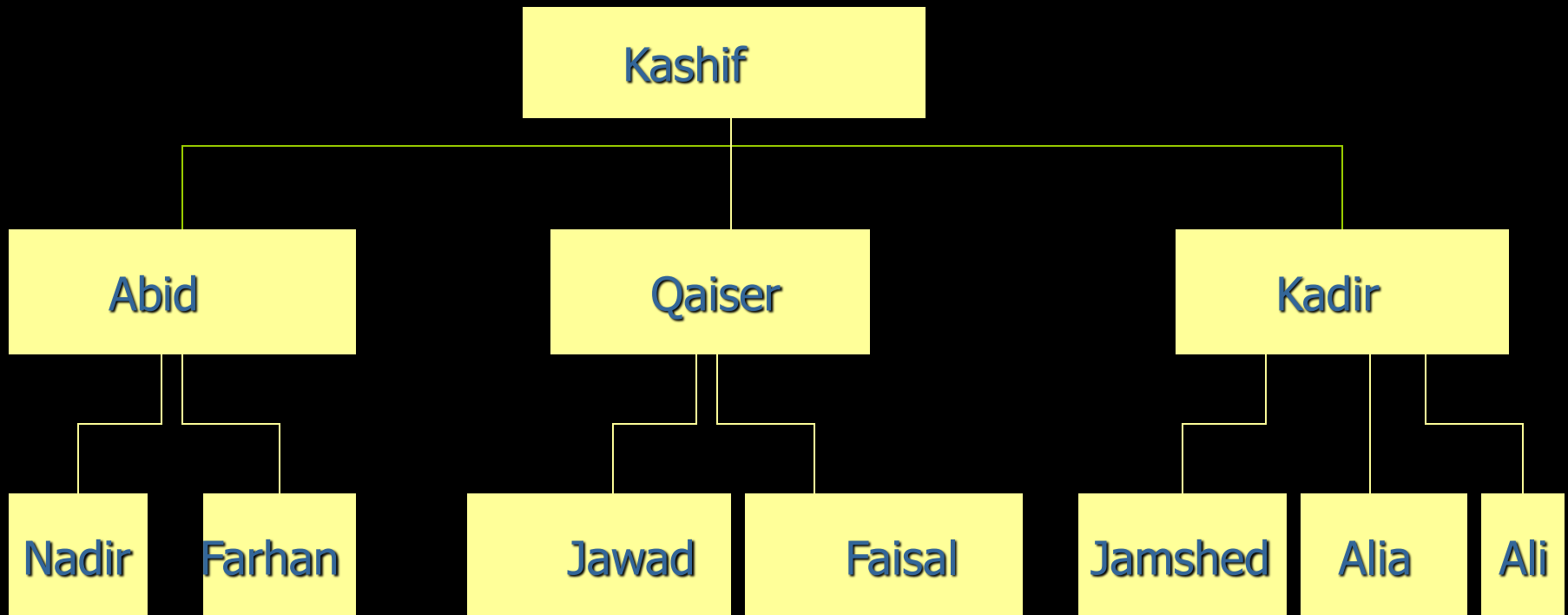
Efficiency of Recursion

- In general, a non recursive version of a program will execute more efficiently in terms of **time** and **space** than a recursive version. This is because the overhead involved in **entering** and **exiting** a **new block (system stack)** is avoided in the non recursive version.
- However, sometimes a recursive solution is the most **natural** and **logical** way of solving a problem as we will soon observe while studying different **Tree data structures**.

Trees

Tree Data Structures

- There are a number of applications where linear data structures are not appropriate. Consider a genealogy (family tree) tree of a family.



Tree Data Structure

- A **linear linked list** will not be able to capture the tree-like relationship with ease.
- Shortly, we will see that for applications that require searching, linear data structures are not suitable.
- We will focus our attention on *binary trees*.

■ Theorem:

A **Tree** with **n** vertices (nodes) has **n-1** edges.

Binary Tree

- A *Binary Tree* is a finite set of elements that is either empty or is partitioned into **three** disjoint subsets. The **first** subset contains a single element called a **root** of the tree. The **other two** subsets are themselves binary trees, called the *left* and *right* sub trees of the original tree.
- A **left** or **right** sub tree can be **empty**.
- Each element of a **binary tree** is called a **node** of the tree.

- Following is an example of **binary tree**. This tree consists of **nine** nodes with **A** as its root. Its **left** sub tree is rooted at **B** and its **right** sub tree is rooted at **C**.

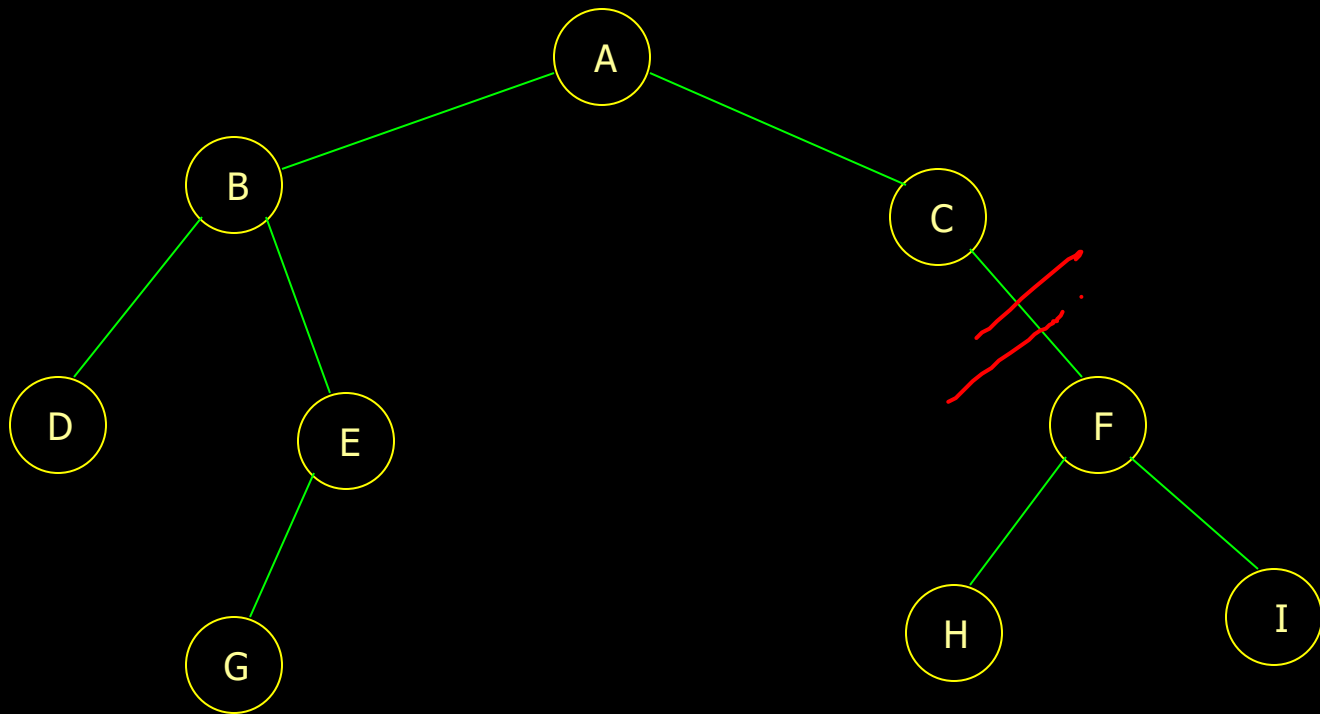
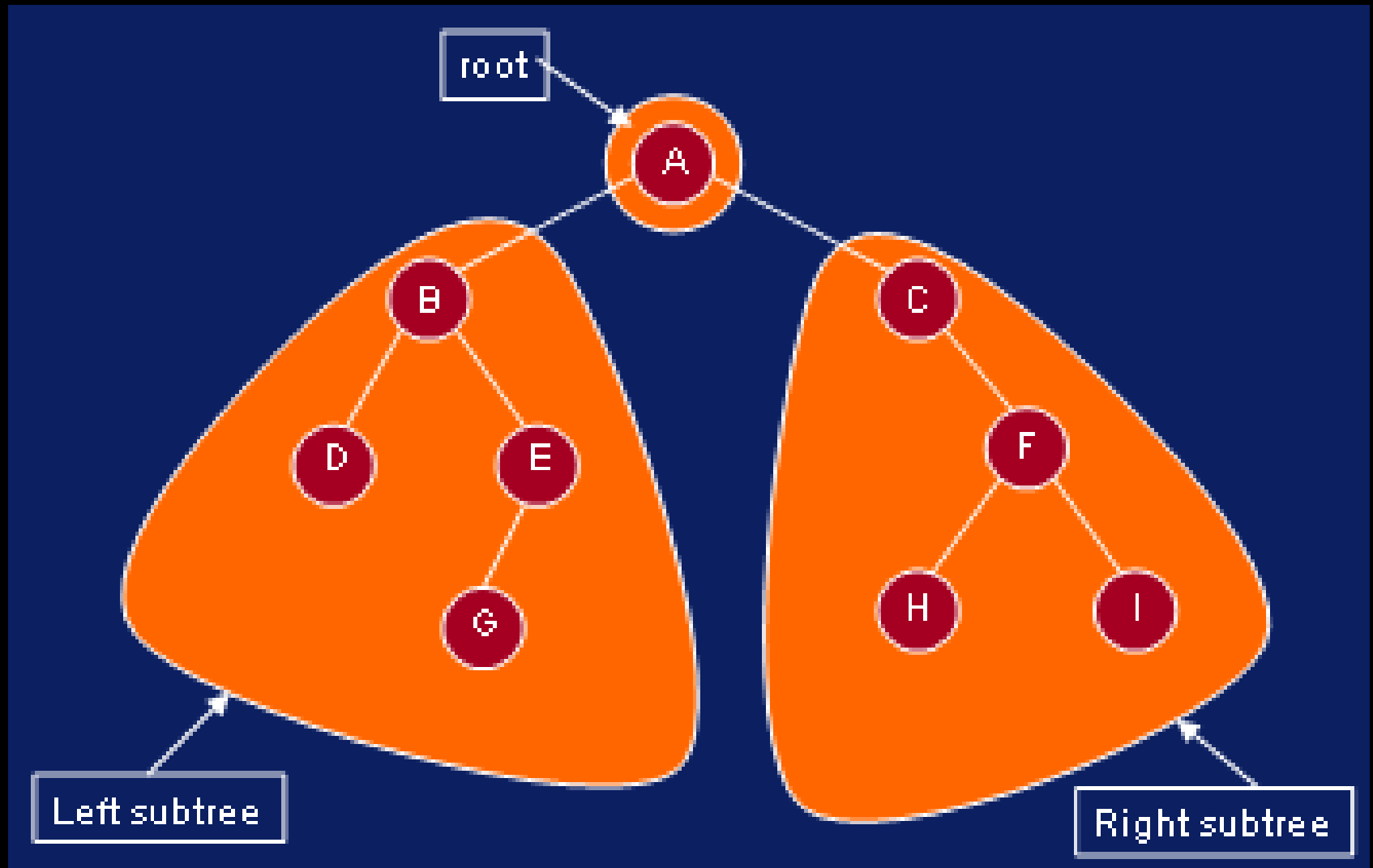


Figure 1

Binary Tree



- The **absence** of a branch indicates any *empty sub tree*. For example in previous figure, the **left** sub tree of the **binary tree** rooted at **C** is **empty**.
- Figures below shows that are ***not Binary Trees***.

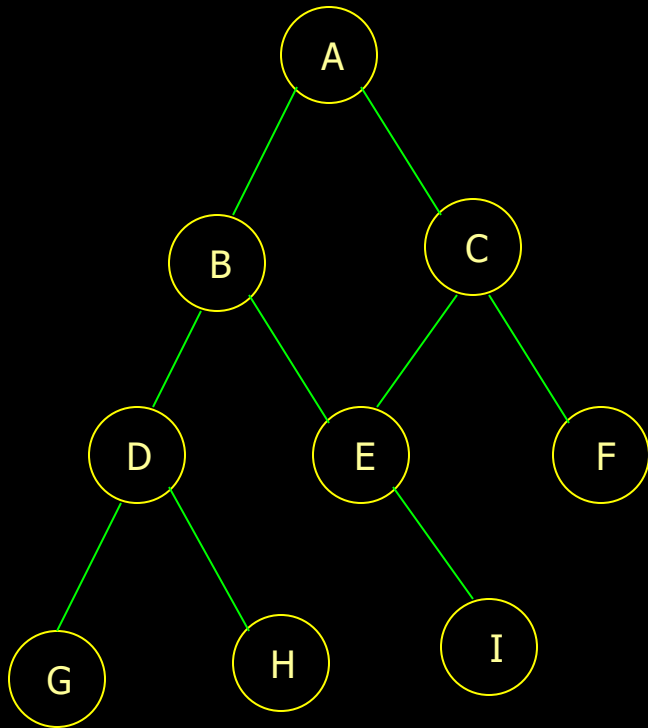


Figure 2

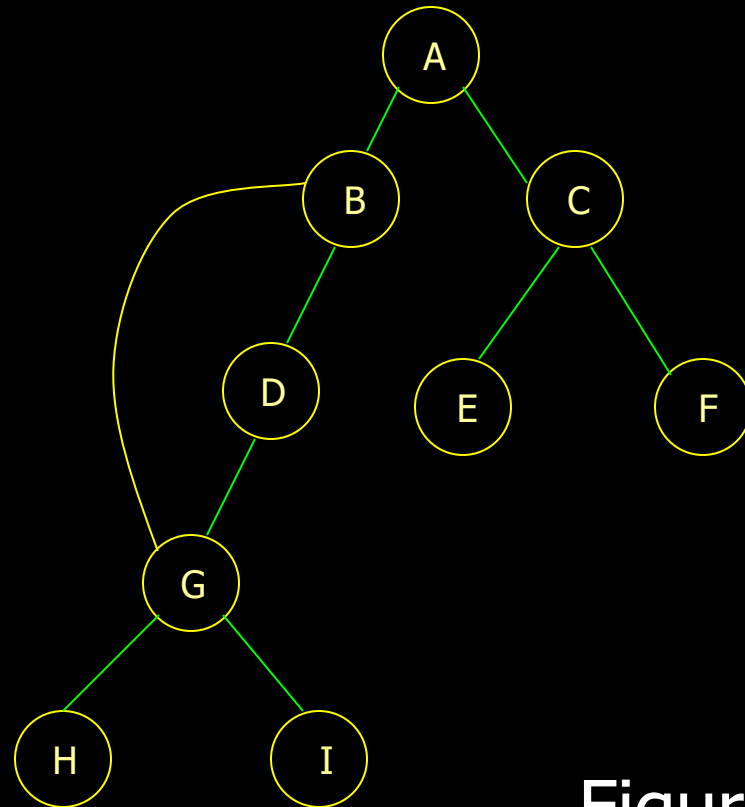


Figure 3

- If **A** is the root of binary tree and **B** is the root of its **left** or **right** sub tree, then **A** is said to be **father** or **parent** of **B** and **B** is said to be **left** or **right son** (**child**) of **A**.
- A node that has no child (son) is called *leaf*.
- Node **n1** is an **ancestor** of node **n2** (and **n2** is a **descendent** of **n1**) if **n1** is either the **parent** of **n2** or the parent of *some ancestor* of **n2**. For example in previous **Figure 1** **A** is an **ancestor** of **G** and **H** is a **descendent** of **C**, but **E** is neither a **descendent** nor **ancestor** of **C**.

- If **every non leaf node** in a binary tree has non empty left and right sub trees, the tree is termed as **strictly binary tree**. Following **Figure 4** is strictly binary tree while **Figure 1** is not.
- A **strictly binary tree** with **n** leaves always contain **$2n - 1$** nodes.

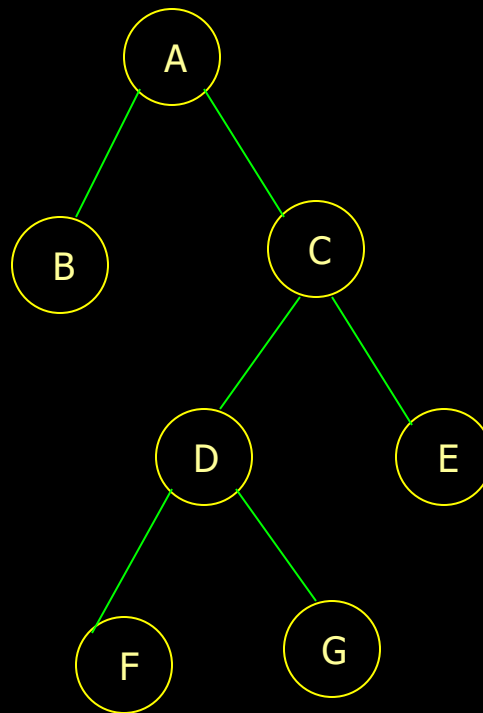
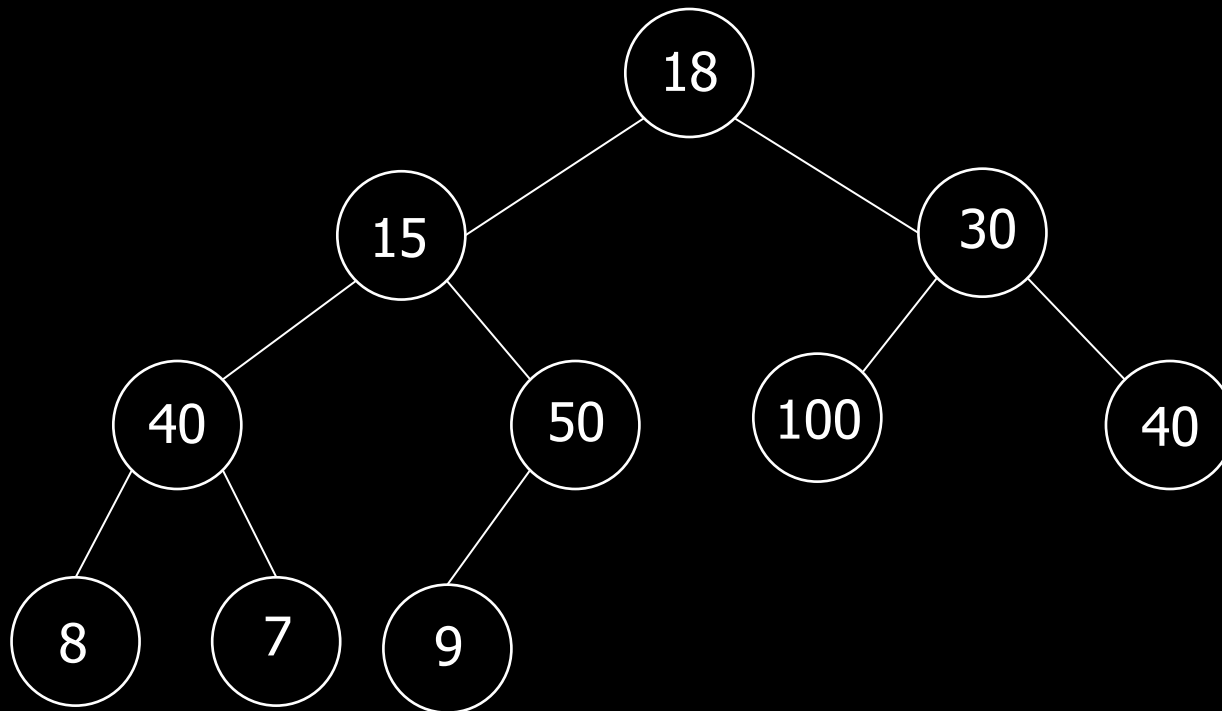


Figure 4

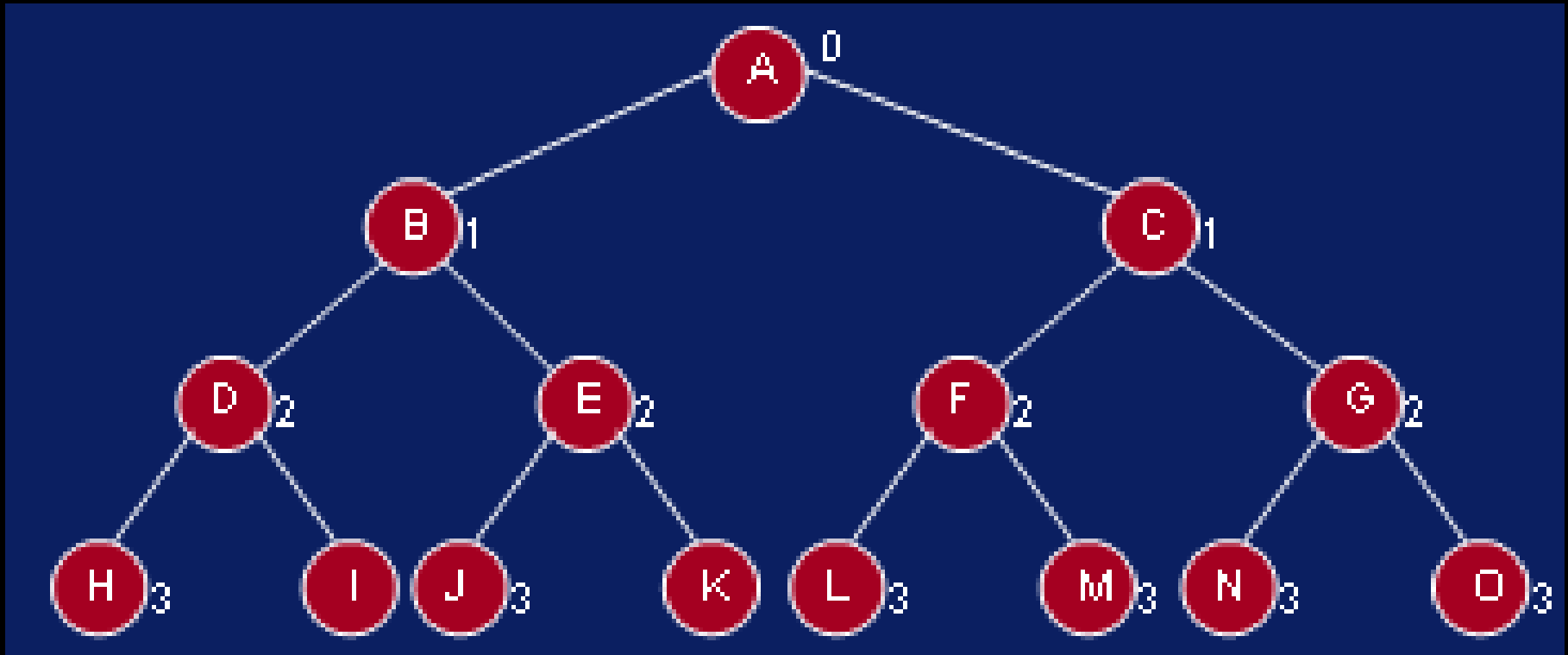
- The **level** of a node in a binary tree is defined as follows :-> The **root** of the tree has **level 0**, and the level of any other node in the tree is **one more** than the level of its **parent**. For example in **Figure 1** node **E** is at **level 2** and node **H** is at **level 3**.
- The **depth** of a binary tree is the **maximum level** of any **leaf** in the tree. Thus depth of **Figure 1** is **3**.
- A **Complete Binary Tree** ,is a binary tree in which each level of the tree is completely filled except possibly the bottom level, and in this level the ***nodes are in the left most positions***. For example,

A Complete Binary Tree



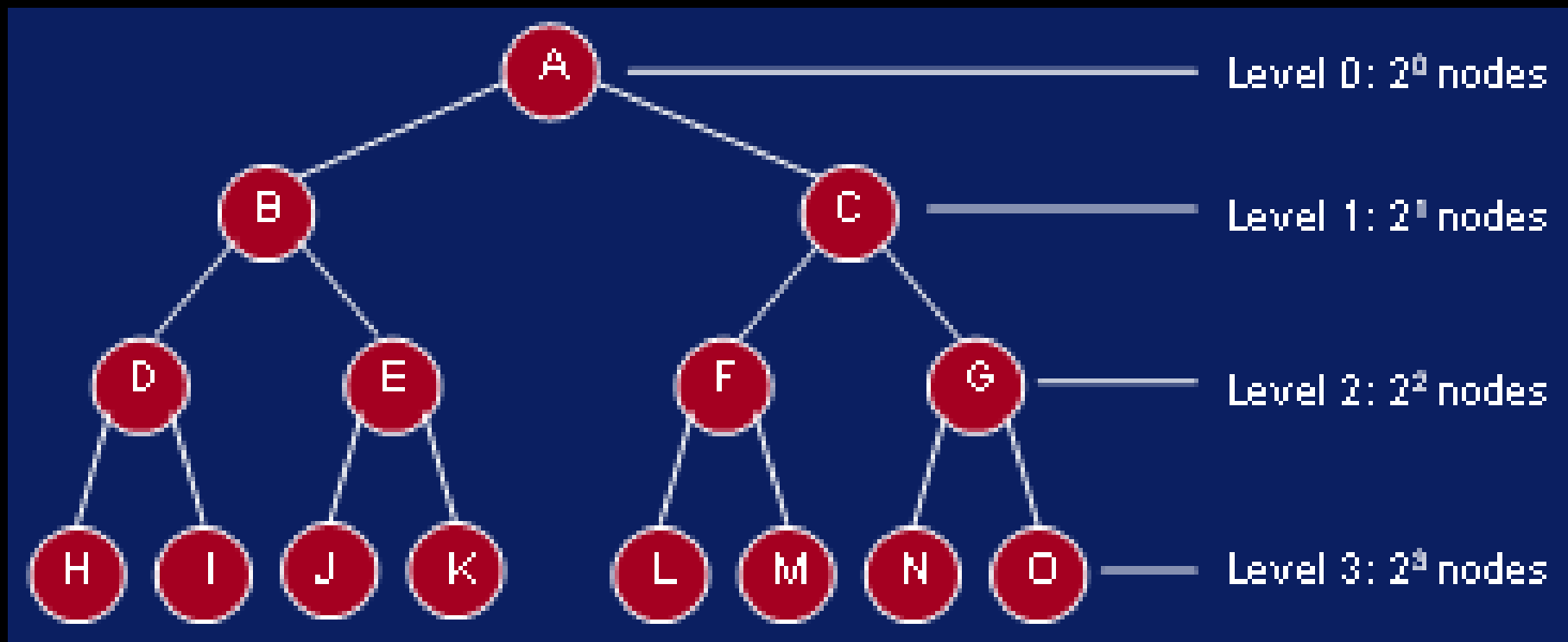
Complete Binary Tree

- A *complete binary tree* of depth d is the **strictly Complete binary** if all of whose leaves are at level d .

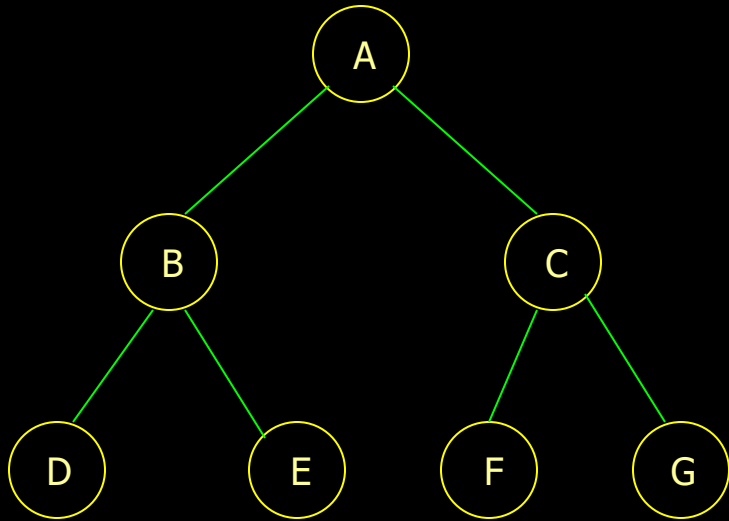


Complete Binary Tree

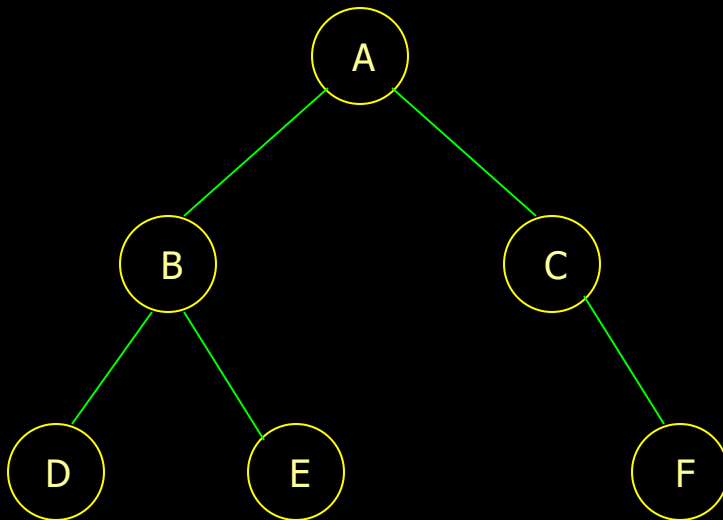
- A *complete binary tree* of depth d is the **strictly binary** all of whose leaves are at level d .



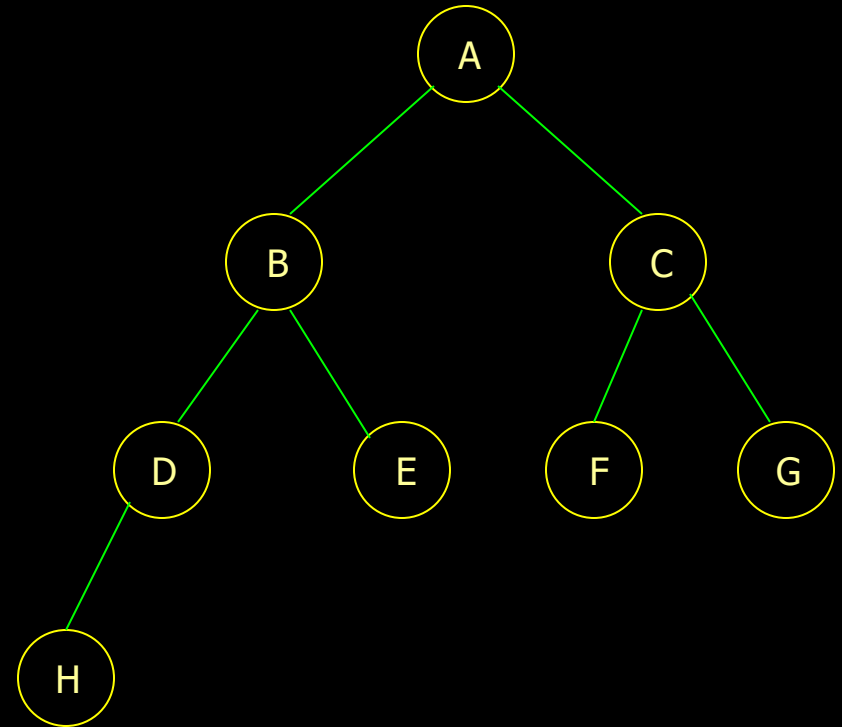
Strictly Complete Binary Tree



Not Complete Binary Tree



Complete Binary Tree



- Another common property is to **traverse** a binary tree i.e. to pass through the tree, i.e. enumerating or visiting each of its nodes once.

Thank You.....